

privateSQL_tpch_eval_supplier_policy

January 7, 2026

1 PrivateSQL + TPC-H Evaluation (Supplier Policy)

This notebook evaluates PrivateSQL-style differential privacy on TPC-H queries using the supplier policy. It builds Spark tables from raw TPC-H .tbl files, constructs protected and public session views, runs DP query variants (Q1, Q4, Q16), and computes error statistics across multiple DP runs.

```
[1]: import os
os.environ["JAVA_HOME"] = "/opt/homebrew/opt/openjdk@17/libexec/openjdk.jdk/
↳Contents/Home"
os.environ["SPARK_HOME"] = "/Users/kinchintong/project/tumult-labs/spark-4.0.
↳1-bin-hadoop3"
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import findspark
findspark.init(os.environ["SPARK_HOME"])
from math import floor
from pyspark import SparkFiles
from pyspark.sql import SparkSession, types as T, functions as F
from tmlt.analytics.privacy_budget import PureDPBudget
from tmlt.analytics.protected_change import (AddOneRow, AddMaxRows,
↳AddRowsWithID)
from tmlt.analytics.query_builder import QueryBuilder
from tmlt.analytics.session import Session, ColumnType
from tmlt.analytics.keyset import KeySet
from functools import reduce
from tmlt.analytics import (
    AddOneRow,
    PureDPBudget,
    QueryBuilder,
    Session,
    AddRowsWithID,
    GroupbyCountQuery,
)

from tmlt.analytics.truncation_strategy import TruncationStrategy
```

```

# Silence some Spark warnings
import warnings
warnings.simplefilter('ignore', UserWarning)
warnings.simplefilter('ignore', ResourceWarning)

findspark.init()

# Set up Spark
spark = (
    SparkSession.builder
        .appName("tpch-setup")
        .master("local[*]")          # drop if connecting to a cluster
        .config("spark.sql.adaptive.enabled", "true")
        .config("spark.driver.memory", "16g")
        .config("spark.executor.memory", "16g")
        .config("spark.driver.maxResultSize", "2g")
        .getOrCreate()
)
print("Spark version:", spark.version)

spark.sparkContext.setLogLevel("ERROR")

spark.conf.set("spark.sql.session.timeZone", "UTC")

# CREATE EMPTY TABLES (schema only) ----
def create_empty_tables():
    for t in TABLES:
        empty = spark.createDataFrame([], SCHEMAS[t])
        # managed Parquet tables; change to .format("delta") if using Delta
        # empty.write.mode("overwrite").format("parquet").saveAsTable(t)
        print(f"Created empty schema in database `{DB}`.")

# LOAD FROM CSV / .tbl (pipe-delimited with trailing '/') ----
def load_from_csv(src_dir: str):
    for t in TABLES:
        schema = SCHEMAS[t]
        path = f"{src_dir}/{t}.tbl"    # change to .csv if needed
        #only if the path exists
        if not os.path.exists(path):
            print(f"Path does not exist: {path}")
            continue

        print(f"Loading {t} from {path}...")

        # create a temporary wide schema of string columns (named) to absorb
        # trailing '/'

```

```

n_cols = len(schema.fields) + 1
tmp_fields = [T.StructField(f"_c{i}", T.StringType(), True) for i in
↳range(n_cols)]
tmp_schema = T.StructType(tmp_fields)
#print("tmp_schema:", tmp_schema)
df_raw = (spark.read
    .option("sep", "|")
    .option("header", "false")
    .option("mode", "PERMISSIVE")
    .schema(tmp_schema) # read wide to absorb trailing '/'
    .csv(path))
#print(f"Raw schema for {t}:")
#df_raw.printSchema()

# keep only first N columns and cast to target schema
cols = df_raw.columns[:len(schema.fields)]
df = df_raw.select([F.col(c).alias(schema.fields[i].name) for i, c in
↳enumerate(cols)])
# cast columns to target types
for f in schema.fields:
    if isinstance(f.dataType, T.DateType):
        df = df.withColumn(f.name, F.to_date(F.col(f.name),
↳"yyyy-MM-dd"))
    elif isinstance(f.dataType, T.DecimalType):
        df = df.withColumn(f.name, F.col(f.name).cast(f.dataType))
    elif isinstance(f.dataType, T.IntegerType):
        df = df.withColumn(f.name, F.col(f.name).cast("int"))
    # strings need no cast
df.write.mode("overwrite").format("parquet").saveAsTable(t)
print(f"Loaded {t} from {path}")

```

WARNING: Using incubator modules: jdk.incubator.vector

Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties

26/01/07 22:24:30 WARN Utils: Your hostname, kinchins-MacBook-Pro.local, resolves to a loopback address: 127.0.0.1; using 192.168.4.184 instead (on interface en0)

26/01/07 22:24:30 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another address

Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties

Setting default log level to "WARN".

To adjust logging level use `sc.setLogLevel(newLevel)`. For SparkR, use `setLogLevel(newLevel)`.

26/01/07 22:24:30 WARN NativeCodeLoader: Unable to load native-hadoop library for your platform... using builtin-java classes where applicable

26/01/07 22:24:30 WARN Utils: Service 'SparkUI' could not bind on port 4040. Attempting port 4041.

Spark version: 4.0.1

```
[2]: DB = "tpch"                                # change as you like
      OUTPUT="supplier_policy_results" # output directory for results
```

```
[3]: # Define canonical TPC-H schemas ----
      dec = lambda p, s: T.DecimalType(p, s)
      SCHEMAS = {
          "region": T.StructType([
              T.StructField("r_regionkey", T.IntegerType(), False),
              T.StructField("r_name", T.StringType(), False),
              T.StructField("r_comment", T.StringType(), True),
          ]),
          "nation": T.StructType([
              T.StructField("n_nationkey", T.IntegerType(), False),
              T.StructField("n_name", T.StringType(), False),
              T.StructField("n_regionkey", T.IntegerType(), False),
              T.StructField("n_comment", T.StringType(), True),
          ]),
          "part": T.StructType([
              T.StructField("p_partkey", T.IntegerType(), False),
              T.StructField("p_name", T.StringType(), False),
              T.StructField("p_mfgr", T.StringType(), False),
              T.StructField("p_brand", T.StringType(), False),
              T.StructField("p_type", T.StringType(), False),
              T.StructField("p_size", T.IntegerType(), False),
              T.StructField("p_container", T.StringType(), False),
              T.StructField("p_retailprice", T.FloatType(), False),
              T.StructField("p_comment", T.StringType(), True),
          ]),
          "supplier": T.StructType([
              T.StructField("s_suppkey", T.IntegerType(), False),
              T.StructField("s_name", T.StringType(), False),
              T.StructField("s_address", T.StringType(), False),
              T.StructField("s_nationkey", T.IntegerType(), False),
              T.StructField("s_phone", T.StringType(), False),
              T.StructField("s_acctbal", T.FloatType(), False),
              T.StructField("s_comment", T.StringType(), True),
          ]),
          "partsupp": T.StructType([
              T.StructField("ps_partkey", T.IntegerType(), False),
              T.StructField("ps_suppkey", T.IntegerType(), False),
              T.StructField("ps_availqty", T.IntegerType(), False),
              T.StructField("ps_supplycost", T.FloatType(), False),
              T.StructField("ps_comment", T.StringType(), True),
          ]),
          "customer": T.StructType([
              T.StructField("c_custkey", T.IntegerType(), False),
              T.StructField("c_name", T.StringType(), False),
```

```

        T.StructField("c_address", T.StringType(), False),
        T.StructField("c_nationkey", T.IntegerType(), False),
        T.StructField("c_phone", T.StringType(), False),
        T.StructField("c_acctbal", T.FloatType(), False),
        T.StructField("c_mktsegment", T.StringType(), True),
        T.StructField("c_comment", T.StringType(), True),
    ]),
    "orders": T.StructType([
        T.StructField("o_orderkey", T.IntegerType(), False),
        T.StructField("o_custkey", T.IntegerType(), False),
        T.StructField("o_orderstatus", T.StringType(), False),
        T.StructField("o_totalprice", T.FloatType(), False),
        T.StructField("o_orderdate", T.DateType(), False),
        T.StructField("o_orderpriority", T.StringType(), False),
        T.StructField("o_clerk", T.StringType(), False),
        T.StructField("o_shippriority", T.IntegerType(), False),
        T.StructField("o_comment", T.StringType(), True),
    ]),
    "lineitem": T.StructType([
        T.StructField("l_orderkey", T.IntegerType(), False),
        T.StructField("l_partkey", T.IntegerType(), False),
        T.StructField("l_suppkey", T.IntegerType(), False),
        T.StructField("l_linenum", T.IntegerType(), False),
        T.StructField("l_quantity", T.FloatType(), False),
        T.StructField("l_extendedprice", T.FloatType(), False),
        T.StructField("l_discount", T.FloatType(), False),
        T.StructField("l_tax", T.FloatType(), False),
        T.StructField("l_returnflag", T.StringType(), False),
        T.StructField("l_linestatus", T.StringType(), False),
        T.StructField("l_shipdate", T.DateType(), False),
        T.StructField("l_commitdate", T.DateType(), False),
        T.StructField("l_receiptdate", T.DateType(), False),
        T.StructField("l_shipinstruct", T.StringType(), False),
        T.StructField("l_shipmode", T.StringType(), False),
        T.StructField("l_comment", T.StringType(), True),
    ]),
}
TABLES = _
↳ ["region", "nation", "part", "supplier", "partsupp", "customer", "orders", "lineitem"]
spark.sql(f"CREATE DATABASE IF NOT EXISTS {DB}")
spark.catalog.setCurrentDatabase(DB)

create_empty_tables()

```

Created empty schema in database `tpch`.

```
[8]: SRC_CSV_DIR = "./raw"      # where *.tbl live
load_from_csv(SRC_CSV_DIR)

# Quick smoke test:
spark.sql("SHOW TABLES").show(truncate=False)
# show record count for each table
for t in TABLES:
    print(f"Table {t} has {spark.sql(f'SELECT count(*) FROM {t}').
    ↪collect()[0][0]} records.")
```

```
Loading region from ./raw/region.tbl...
Loaded region from ./raw/region.tbl
Loading nation from ./raw/nation.tbl...
Loaded nation from ./raw/nation.tbl
Loading part from ./raw/part.tbl...
```

```
Loaded part from ./raw/part.tbl
Loading supplier from ./raw/supplier.tbl...
Loaded supplier from ./raw/supplier.tbl
Loading partsupp from ./raw/partsupp.tbl...
Loaded partsupp from ./raw/partsupp.tbl
Loading customer from ./raw/customer.tbl...
Loaded customer from ./raw/customer.tbl
Loading orders from ./raw/orders.tbl...
```

```
Loaded orders from ./raw/orders.tbl
Loading lineitem from ./raw/lineitem.tbl...
```

```
Loaded lineitem from ./raw/lineitem.tbl
```

```
+-----+-----+-----+
|namespace|tableName|isTemporary|
+-----+-----+-----+
|tpch     |customer |false      |
|tpch     |lineitem |false      |
|tpch     |nation   |false      |
|tpch     |orders   |false      |
|tpch     |part     |false      |
|tpch     |partsupp |false      |
|tpch     |region   |false      |
|tpch     |supplier |false      |
+-----+-----+-----+
```

```
Table region has 5 records.
Table nation has 25 records.
Table part has 200000 records.
```

Table supplier has 10000 records.
Table partsupp has 800000 records.
Table customer has 150000 records.
Table orders has 1500000 records.
Table lineitem has 6001215 records.

1.1 TPC-H Queries Evaluated

- **Q1 - Pricing Summary Report Query** This query reports the amount of business that was billed, shipped, and returned.
- **Q4 - Order Priority Checking Query** This query determines how well the order priority system is working and gives an assessment of customer satisfaction.
- **Q13 - Customer Distribution Query** This query seeks relationships between customers and the size of their orders.
- **Q16 - Parts/Supplier Relationship Query** This query finds out how many suppliers can supply parts with given attributes. It might be used, for example, to determine whether there is a sufficient number of suppliers for heavily ordered parts.

1.2 PrivateSQL paper evaluation

1.2.1 Dataset: TPC-H

1.2.2 Privacy Policy: Customer

1.2.3 Privacy Budget : 1.0

privacy model: customer table is private orders table have foreign key of customer table (custkey)
lineitem table have foreign key of orders table (orderkey)

```
[9]: SRC_CSV_DIR = "./raw"
dataframes = {}

TABLES = [
    "region", "nation", "part", "supplier", "partsupp", "customer", "orders", "lineitem"
]
for table_name in TABLES:
    path = f"{SRC_CSV_DIR}/{table_name}.tbl"
    schema = SCHEMAS[table_name]
    dataframes[table_name] = (spark.read
                              .option("sep", "|")
                              .option("header", "false")
                              .option("mode", "PERMISSIVE")
                              .schema(schema)
                              .csv(path))

#needed for q1
lineitem_with_disc_view = (dataframes["lineitem"]
```

```

        .withColumn("disc_price",
                    (dataframes["lineitem"]["l_extendedprice"].cast("double") * (1.
↪0 - dataframes["lineitem"]["l_discount"].cast("double"))))
        .withColumn("charge",
                    (dataframes["lineitem"]["l_extendedprice"].cast("double") * (1.
↪0 - dataframes["lineitem"]["l_discount"].cast("double")) * (1.0 + ↪
↪dataframes["lineitem"]["l_tax"].cast("double"))))
    )
    dataframes["lineitem_with_disc"] = lineitem_with_disc_view

WORD1 = 'special'
WORD2 = 'requests'
q13_view = f"""
SELECT
    c.c_custkey,
    COUNT(o.o_orderkey) AS c_count
FROM customer c
LEFT JOIN orders o
    ON o.o_custkey = c.c_custkey
    AND o.comment NOT LIKE '%{WORD1}%{WORD2}%'
GROUP BY c.c_custkey;
"""

V_q13 = spark.sql(q13_view)
dataframes["V_q13"] = V_q13

for table_name in dataframes:
    print(f"Schema for table: {table_name}")
    dataframes[table_name].show(5)

budget = PureDPBudget(float("inf"))
#budget = PureDPBudget(3)

id_space = "supplier_policy_id_space"

session = (
    Session.Builder()
        .with_privacy_budget(budget)
        .with_id_space(id_space)
        .with_private_dataframe(
            "lineitem_with_disc",
            dataframes["lineitem_with_disc"],
            protected_change=AddMaxRows(60080),
        )
)

```

```

.with_private_dataframe(
    "lineitem",
    dataframes["lineitem"],
    protected_change=AddMaxRows(60080),
)
.with_public_dataframe(
    "orders",
    dataframes["orders"],
)
.with_public_dataframe(
    "customer",
    dataframes["customer"],
)
.with_public_dataframe(
    "region",
    dataframes["region"],
)
.with_public_dataframe(
    "nation",
    dataframes["nation"],
)
.with_public_dataframe(
    "part",
    dataframes["part"],
)
.with_public_dataframe(
    "V_q13",
    dataframes["V_q13"],
)
.with_private_dataframe(
    "supplier",
    dataframes["supplier"],
    protected_change=AddOneRow(),
)
.with_private_dataframe(
    "partsupp",
    dataframes["partsupp"],
    protected_change=AddMaxRows(80),
)
.build()
)

```

Schema for table: region

r_regionkey	r_name	r_comment
0	AFRICA	lar deposits. bli...
1	AMERICA	hs use ironic, ev...

2	ASIA	ges. thinly even ...
3	EUROPE	ly final courts c...
4	MIDDLE EAST	uickly special ac...

Schema for table: nation

n_nationkey	n_name	n_regionkey	n_comment
0	ALGERIA	0	haggle. carefull...
1	ARGENTINA	1	al foxes promise ...
2	BRAZIL	1	y alongside of th...
3	CANADA	1	eas hang ironic, ...
4	EGYPT	4	y above the caref...

only showing top 5 rows

Schema for table: part

p_partkey	p_name	p_mfgr	p_brand	p_type	p_size	p_container	p_retailprice	p_comment
1	goldenrod lavende...	Manufacturer#1	Brand#13	PROMO	BURNISHED C...			
7	JUMBO PKG	901.0	ly. slyly ironi					
2	blush thistle blu...	Manufacturer#1	Brand#13	LARGE	BRUSHED BRASS			
1	LG CASE	902.0	lar accounts amo					
3	spring green yell...	Manufacturer#4	Brand#42	STANDARD	POLISHED...			
21	WRAP CASE	903.0	egular deposits hag					
4	cornflower chocol...	Manufacturer#3	Brand#34	SMALL	PLATED BRASS			
14	MED DRUM	904.0	p furiously r					
5	forest brown cora...	Manufacturer#3	Brand#32	STANDARD	POLISHED...			
15	SM PKG	905.0	wake carefully					

only showing top 5 rows

Schema for table: supplier

s_suppkey	s_name	s_address	s_nationkey	s_phone	s_acctbal	s_comment
1	Supplier#000000001	N kD4on90M Ipw3,...	17	27-918-335-1736		
		5755.94 each slyly above ...				
2	Supplier#000000002	89eJ5ksX3ImxJQBvx...	5	15-679-861-2259		
		4032.68 slyly bold instr...				
3	Supplier#000000003	q1,G3Pj60jIuUYfUo...	1	11-383-516-1199		

```

4192.4|blithely silent r...|
|      4|Supplier#000000004|Bk7ah4CK8SYQTepEm...|      15|25-843-787-7479|
4641.08|riously even requ...|
|      5|Supplier#000000005|  Gcdm2rJRzl5qlTVzc|      11|21-151-690-3663|
-283.84|. slyly regular p...|
+-----+-----+-----+-----+-----+-----+
+-----+-----+
only showing top 5 rows
Schema for table: partsupp
+-----+-----+-----+-----+-----+-----+
|ps_partkey|ps_suppkey|ps_availqty|ps_supplycost|      ps_comment|
+-----+-----+-----+-----+-----+-----+
|      1|      2|      3325|      771.64|, even theodolite...|
|      1|     2502|      8076|      993.49|ven ideas. quickl...|
|      1|     5002|      3956|      337.09|after the fluffil...|
|      1|     7502|      4069|      357.84|al, regular depen...|
|      2|      3|      8895|      378.49|nic accounts. fin...|
+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
Schema for table: customer
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|c_custkey|      c_name|      c_address|c_nationkey|
c_phone|c_acctbal|c_mktsegment|      c_comment|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      1|Customer#000000001|  IVhzIApeRb ot,c,E|      15|25-989-741-2988|
711.56|  BUILDING|to the even, regu...|
|      2|Customer#000000002|XSTf4,NCwDVaWNe6t...|      13|23-768-687-3665|
121.65|  AUTOMOBILE|l accounts. blith...|
|      3|Customer#000000003|      MG9kdTD2WBHm|      1|11-719-748-3364|
7498.12|  AUTOMOBILE| deposits eat sly...|
|      4|Customer#000000004|      XxVSJsLAGtn|      4|14-128-190-5944|
2866.83|  MACHINERY| requests. final,...|
|      5|Customer#000000005|KvpyuHCplrB84WgAi...|      3|13-750-942-6364|
794.47|  HOUSEHOLD|n accounts will h...|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
only showing top 5 rows
Schema for table: orders
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|o_orderkey|o_custkey|o_orderstatus|o_totalprice|o_orderdate|o_orderpriority|
o_clerk|o_shippriority|      o_comment|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+
|      1|     36901|      0|  173665.47| 1996-01-02|
5-LOW|Clerk#000000951|      0|nstructions sleep...|

```

2	78002	0	46929.18	1996-12-01
1-URGENT	Clerk#000000880	0	foxes. pending a...	
3	123314	F	193846.25	1993-10-14
5-LOW	Clerk#000000955	0	sly final account...	
4	136777	0	32151.78	1995-10-11
5-LOW	Clerk#000000124	0	sits. slyly regul...	
5	44485	F	144659.2	1994-07-30
5-LOW	Clerk#000000925	0	quickly. bold dep...	

only showing top 5 rows
Schema for table: lineitem

l_orderkey	l_partkey	l_suppkey	l_linenum	l_quantity	l_extendedprice	l_discount	l_tax	l_returnflag	l_linestatus	l_shipdate	l_commitdate	l_receiptdate	l_shipinstruct	l_shipmode	l_comment
1	155190	7706	1	17.0	21168.23	0.04	0.02	N	0	1996-03-13	1996-02-12	1996-03-22	DELIVER IN PERSON	TRUCK	regular courts abo...
1	67310	7311	2	36.0	45983.16	0.09	0.06	N	0	1996-04-12	1996-02-28	1996-04-20	TAKE BACK RETURN	MAIL	ly final dependen...
1	63700	3701	3	8.0	13309.6	0.1	0.02	N	0	1996-01-29	1996-03-05	1996-01-31	TAKE BACK RETURN	REG AIR	riously. regular,...
1	2132	4633	4	28.0	28955.64	0.09	0.06	N	0	1996-04-21	1996-03-30	1996-05-16	NONE	AIR	lites. fluffily e...
1	24027	1534	5	24.0	22824.48	0.1	0.04	N	0	1996-03-30	1996-03-14	1996-04-01	NONE	FOB	pending foxes. s...

only showing top 5 rows
Schema for table: lineitem_with_disc

l_orderkey	l_partkey	l_suppkey	l_linenum	l_quantity	l_extendedprice	l_discount	l_tax	l_returnflag	l_linestatus	l_shipdate	l_commitdate	l_receiptdate	l_shipinstruct	l_shipmode	l_comment	disc_price
------------	-----------	-----------	-----------	------------	-----------------	------------	-------	--------------	--------------	------------	--------------	---------------	----------------	------------	-----------	------------

```

charge|
+-----+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
+
|      1| 155190| 7706|      1| 17.0| 21168.23|
0.04| 0.02|      N|      0|1996-03-13| 1996-02-12|
1996-03-22|DELIVER IN PERSON| TRUCK|egular courts
abo...|20321.501268925873|20727.931285219973|
|      1| 67310| 7311|      2| 36.0| 45983.16|
0.09| 0.06|      N|      0|1996-04-12| 1996-02-28| 1996-04-20| TAKE
BACK RETURN| MAIL|ly final
dependen...|41844.675577738904|44355.356056285156|
|      1| 63700| 3701|      3| 8.0| 13309.6|
0.1| 0.02|      N|      0|1996-01-29| 1996-03-05| 1996-01-31| TAKE
BACK RETURN| REG AIR|riously.
regular,...|11978.639628604651|12218.212415821876|
|      1| 2132| 4633|      4| 28.0| 28955.64|
0.09| 0.06|      N|      0|1996-04-21| 1996-03-30| 1996-05-16|
NONE| AIR|lites. fluffily e...| 26349.63286519656| 27930.61080177074|
|      1| 24027| 1534|      5| 24.0| 22824.48|
0.1| 0.04|      N|      0|1996-03-30| 1996-03-14| 1996-04-01|
NONE| FOB| pending foxes. s...|20542.032387863874| 21363.71366501242|
+-----+-----+-----+-----+-----+-----+-----+
--+-----+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
+
only showing top 5 rows
Schema for table: V_q13
+-----+-----+
|c_custkey|c_count|
+-----+-----+
|      148|      21|
|      463|      18|
|      471|       0|
|      496|      21|
|      833|      13|
+-----+-----+
only showing top 5 rows

```

```

[10]: def query_dp_eval(eps: PureDPBudget, session: Session, query_num, query_dp_run,
    num_runs: int = 1, start_run: int = 0):
    for run in range(start_run, num_runs):
        print(f"Running Q{query_num} DP run {run+1}/{num_runs} with epsilon={eps.
    epsilon}...")
        runx = query_dp_run(eps, session)

```

```

output_file = f"./output/{OUTPUT}/q{query_num}/
↳q{query_num}_dp_result_eps_{eps.epsilon}_runs_{run+1}_csv"
print(output_file)
runx.coalesce(1).write \
    .mode("overwrite") \
    .option("header", "true") \
    .csv(output_file)

```

1.2.4 Q1 — Pricing Summary Report

```

SELECT
    l_returnflag,
    l_linestatus,
    SUM(l_quantity) AS sum_qty,
    SUM(l_extendedprice) AS sum_base_price,
    SUM(l_extendedprice * (1 - l_discount)) AS sum_disc_price,
    SUM(l_extendedprice * (1 - l_discount) * (1 + l_tax)) AS sum_charge,
    AVG(l_quantity) AS avg_qty,
    AVG(l_extendedprice) AS avg_price,
    AVG(l_discount) AS avg_disc,
    COUNT(_) AS count_order
FROM lineitem
WHERE l_shipdate <= DATE '1998-12-01' - INTERVAL '[DELTA]' DAY
GROUP BY l_returnflag, l_linestatus
ORDER BY l_returnflag, l_linestatus;

```

```
[11]: def q1_dp_run(eps: PureDPBudget, session: Session):
```

```

    #1) Create V_q1 with casts + derived columns
    DELTA = 90

    q1_view = (
        QueryBuilder("lineitem_with_disc")
        .select([
            "l_returnflag",
            "l_linestatus",
            "l_quantity",
            "l_extendedprice",
            "l_discount",
            "l_tax",
            "l_shipdate",
            "disc_price",
            "charge",
        ])
    )

```

```

# delete V_q1 only if it exists to avoid errors
try:
    session.delete_view("V_q1")
except Exception as e:
    pass

session.create_view(source_id="V_q1", query_expr=q1_view, cache=True)

#2) Group + filter (use Spark-friendly interval)
groupby_keys = KeySet.from_dict({
    "l_returnflag": ["A", "N", "R"],
    "l_linestatus": ["F", "O"],
})

q1_linear_grouped = (
    QueryBuilder("V_q1")
        .filter(f"l_shipdate <= DATE '1998-12-01' - INTERVAL {DELTA} DAYS")
        .groupby(groupby_keys)
        .count(name="count_order")
)

dp_count_order = session.evaluate(q1_linear_grouped, privacy_budget=eps)
return dp_count_order

```

1.2.5 Q4 - Order Priority Checking Query

```

SELECT
    o_orderpriority,
    COUNT(*) AS order_count
FROM
    orders
WHERE
    o_orderdate >= DATE '[DATE]'
    AND o_orderdate < DATE '[DATE]' + INTERVAL '3' MONTH
    AND EXISTS (
        SELECT
            *
        FROM
            lineitem
        WHERE
            l_orderkey = o_orderkey
            AND l_commitdate < l_receiptdate
    )
GROUP BY
    o_orderpriority
ORDER BY

```

o_orderpriority;

```
[12]: def q4_dp_run(eps: PureDPBudget, session: Session):
    print("Building Q4 deduped view V_q4...")
    from tmlt.analytics.query_builder import QueryBuilder

    q_lineitem_filtered = (
        QueryBuilder("lineitem")
        .filter("l_commitdate < l_receiptdate")
        .select(["l_orderkey"])
        .rename({"l_orderkey": "orderkey"})
    )

    # orders columns (include orderdate so you DON'T need to join orders again,
    # later)
    q_orders = (
        QueryBuilder("orders")
        .select(["o_orderkey", "o_orderpriority", "o_orderdate"])
        .rename({"o_orderkey": "orderkey"})
    )

    orders = dataframes["orders"].withColumnRenamed("o_orderkey", "orderkey")

    # join
    start_date = "1993-07-01"
    q4_view = q_lineitem_filtered.join_public(
        orders,
        join_columns=["orderkey"],
    ).select(["orderkey", "o_orderpriority", "o_orderdate"])
    .filter(
        f"o_orderdate >= DATE '{start_date}' AND "
        f"o_orderdate < DATE '{start_date}' + INTERVAL 3 MONTH"
    )

    # create session view
    try:
        session.delete_view("V_q4_1")
    except Exception:
        pass

    session.create_view(source_id="V_q4_1", query_expr=q4_view, cache=True)

    eps = PureDPBudget(epsilon=1.0)

    groupby_keys = KeySet.from_dict({
```

```

        "o_orderpriority": ["1-URGENT", "2-HIGH", "3-MEDIUM", "4-NOT_
SPECIFIED", "5-LOW"]
    })

    q4_1 = (
        QueryBuilder("V_q4_1")
        .select(["o_orderpriority"])
        .groupby(groupby_keys)
    )

    q4_count = q4_1.count(name="order_count")

    q4_dp_result = session.evaluate(q4_count, privacy_budget=eps)

    return q4_dp_result

```

Query 16

```

[13]: def q16_dp_DropExcess_1_run(eps: PureDPBudget, session: Session):
    from pyspark.sql.functions import col
    q16_view_part = (
        QueryBuilder("part")
        .select(["p_partkey", "p_brand", "p_type", "p_size"])
        .rename({"p_partkey": "partkey"})
    )

    q16_view_partsupp = (
        QueryBuilder("partsupp")
        .select(["ps_partkey", "ps_suppkey"])
        .rename({"ps_partkey": "partkey", "ps_suppkey": "suppkey"})
    )

    q16_view_supplier = (
        QueryBuilder("supplier")
        .select(["s_suppkey", "s_comment"])
        .filter("s_comment NOT LIKE '%Customer%Complaints%'")
        .rename({"s_suppkey": "suppkey"})
    )

    q_16_view_1 = q16_view_partsupp.join_private(
        right_operand=q16_view_supplier,
        join_columns=["suppkey"],
        truncation_strategy_left=TruncationStrategy.DropExcess(1),
        truncation_strategy_right=TruncationStrategy.DropExcess(1),
    )

```

```

part = dataframes["part"].select(
    "p_brand",
    "p_type",
    "p_size",
    col("p_partkey").alias("partkey"),
)

q_16_view = (
    q_16_view_1.join_public(
        part,
        join_columns=["partkey"]
    )
)

try:
    session.delete_view("V_q16")
except Exception:
    pass

session.create_view(source_id="V_q16", query_expr=q_16_view, cache=True)

'''
SELECT
    p_brand, p_type, p_size,
    COUNT(*) AS supplier_cnt
FROM V_q16
WHERE p_brand <> '{BRAND}'
    AND p_type NOT LIKE '{TYPE_PREFIX}%'
    AND p_size IN ({sizes_csv})
GROUP BY p_brand, p_type, p_size
'''

syllable1 = ["STANDARD", "SMALL", "MEDIUM", "LARGE", "ECONOMY", "PROMO"]
syllable2 = ["ANODIZED", "BURNISHED", "PLATED", "POLISHED", "BRUSHED"]
syllable3 = ["TIN", "NICKEL", "BRASS", "STEEL", "COPPER"]

TPC_H_TYPES = [
    f"{s1} {s2} {s3}"
    for s1 in syllable1
    for s2 in syllable2
    for s3 in syllable3
]

groupby_keys = KeySet.from_dict({
    "p_brand": ["Brand#11", "Brand#12", "Brand#13", "Brand#14", "Brand#15",
               "Brand#21", "Brand#22", "Brand#23", "Brand#24", "Brand#25",

```

```

        "Brand#31", "Brand#32", "Brand#33", "Brand#34", "Brand#35",
        "Brand#41", "Brand#42", "Brand#43", "Brand#44", "Brand#45",
        "Brand#51", "Brand#52", "Brand#53", "Brand#54", "Brand#55"],
    "p_type": [
        f"{s1} {s2} {s3}"
        for s1 in syllable1
        for s2 in syllable2
        for s3 in syllable3
    ],
    "p_size": list(range(1, 51)),
})

BRAND = "Brand#12"
TYPE_PREFIX = "LARGE ANODIZED"
SIZES = [3, 6, 9, 12, 18, 24, 36, 48]
sizes_csv = ",".join(map(str, SIZES))

q16_filtered = (
    QueryBuilder("V_q16")
        .filter(f"p_brand <> '{BRAND}' AND p_type NOT LIKE '{TYPE_PREFIX}%'\n
↪AND p_size IN ({sizes_csv})")
        .select(["p_brand", "p_type", "p_size"])
        .groupby(groupby_keys)
        .count(name="supplier_cnt")
)

dp_result = session.evaluate(q16_filtered, privacy_budget=eps)
return dp_result

```

```

[14]: def q16_dp_DropNonUnique_run(eps: PureDPBudget, session: Session):
    from pyspark.sql.functions import col
    q16_view_part = (
        QueryBuilder("part")
            .select(["p_partkey", "p_brand", "p_type", "p_size"])
            .rename({"p_partkey": "partkey"})
    )

    q16_view_partsupp = (
        QueryBuilder("partsupp")
            .select(["ps_partkey", "ps_suppkey"])
            .rename({"ps_partkey": "partkey", "ps_suppkey": "suppkey"})
    )

    q16_view_supplier = (
        QueryBuilder("supplier")
            .select(["s_suppkey", "s_comment"])
            .filter("s_comment NOT LIKE '%Customer%Complaints%'")
    )

```

```

        .rename({"s_suppkey": "suppkey"})
    )

    q_16_view_1 = q16_view_partsupp.join_private(
        right_operand=q16_view_supplier,
        join_columns=["suppkey"],
        truncation_strategy_left=TruncationStrategy.DropNonUnique(),
        truncation_strategy_right=TruncationStrategy.DropNonUnique(),
    )

    part = dataframes["part"].select(
        "p_brand",
        "p_type",
        "p_size",
        col("p_partkey").alias("partkey"),
    )

    q_16_view = (
        q_16_view_1.join_public(
            part,
            join_columns=["partkey"]
        )
    )

    try:
        session.delete_view("V_q16")
    except Exception:
        pass

    session.create_view(source_id="V_q16", query_expr=q_16_view, cache=True)

    '''
    SELECT
        p_brand, p_type, p_size,
        COUNT(*) AS supplier_cnt
    FROM V_q16
    WHERE p_brand <> '{BRAND}'
        AND p_type NOT LIKE '{TYPE_PREFIX}%'
        AND p_size IN ({sizes_csv})
    GROUP BY p_brand, p_type, p_size
    '''

    syllable1 = ["STANDARD", "SMALL", "MEDIUM", "LARGE", "ECONOMY", "PROMO"]
    syllable2 = ["ANODIZED", "BURNISHED", "PLATED", "POLISHED", "BRUSHED"]
    syllable3 = ["TIN", "NICKEL", "BRASS", "STEEL", "COPPER"]

    TPC_H_TYPES = [

```

```

        f"{s1} {s2} {s3}"
        for s1 in syllable1
        for s2 in syllable2
        for s3 in syllable3
    ]

    groupby_keys = KeySet.from_dict({
        "p_brand": ["Brand#11", "Brand#12", "Brand#13", "Brand#14", "Brand#15",
                    "Brand#21", "Brand#22", "Brand#23", "Brand#24", "Brand#25",
                    "Brand#31", "Brand#32", "Brand#33", "Brand#34", "Brand#35",
                    "Brand#41", "Brand#42", "Brand#43", "Brand#44", "Brand#45",
                    "Brand#51", "Brand#52", "Brand#53", "Brand#54", "Brand#55"],
        "p_type": [
            f"{s1} {s2} {s3}"
            for s1 in syllable1
            for s2 in syllable2
            for s3 in syllable3
        ],
        "p_size": list(range(1, 51)),
    })

    BRAND = "Brand#12"
    TYPE_PREFIX = "LARGE ANODIZED"
    SIZES = [3, 6, 9, 12, 18, 24, 36, 48]
    sizes_csv = ",".join(map(str, SIZES))

    q16_filtered = (
        QueryBuilder("V_q16")
        .filter(f"p_brand <> '{BRAND}' AND p_type NOT LIKE '{TYPE_PREFIX}%' ↵
↵AND p_size IN ({sizes_csv})")
        .select(["p_brand", "p_type", "p_size"])
        .groupby(groupby_keys)
        .count(name="supplier_cnt")
    )

    dp_result = session.evaluate(q16_filtered, privacy_budget=eps)
    return dp_result

```

```

[ ]: for run in [1000]:
    eps = PureDPBudget(1)
    query_dp_eval(eps, session, query_num="1", query_dp_run=q1_dp_run, ↵
↵num_runs=run, start_run=1000)
    query_dp_eval(eps, session, query_num="4", query_dp_run=q4_dp_run, ↵
↵num_runs=run, start_run=1000)
    query_dp_eval(eps, session, query_num="16_DropExcess_1", ↵
↵query_dp_run=q16_dp_DropExcess_1_run, num_runs=run, start_run=1000)

```

```

query_dp_eval(eps, session, query_num="16_DropNonUnique",
↳query_dp_run=q16_dp_DropNonUnique_run, num_runs=run, start_run=1000)

```

1.3 Calculate Error

```

[4]: from typing import List
import matplotlib.pyplot as plt
import pandas as pd

def compute_errors_privatesql_style(
    dp_df,
    truth_df,
    keys: List[str],
    metric: str = "count_order",
    run_col: str = "run_id",
    join_type: str = "left",
    truth_missing_as_zero: bool = True,
):
    """
    Compute PrivateSQL-style errors per output group per run.
     $Qerror(y, y\sim) = |y - y\sim|$ 
     $RelError(y, y\sim) = |y - y\sim| / \max(50, y)$ 

    Returns columns:
        run_id,
        keys,
        <metric>_truth,
        <metric>_dp,
        qerror_<metric>,
        relerror_<metric>
    """

    # Prepare DP side
    dp = (
        dp_df
        .withColumn(metric, F.col(metric).cast("double"))
        .withColumnRenamed(metric, f"{metric}_dp")
    )
    #dp.show(5)

    # Prepare truth side
    tr = (
        truth_df
        .withColumn(metric, F.col(metric).cast("double"))

```

```

        .withColumnRenamed(metric, f"{metric}_truth")
    )

    #tr.show(5)

    joined = dp.alias("dp").join(tr.alias("tr"), on=keys, how=join_type)

    if truth_missing_as_zero:
        joined = joined.withColumn(
            f"{metric}_truth",
            F.coalesce(F.col(f"{metric}_truth"), F.lit(0.0))
        )

    # Absolute error |y - y~|
    joined = joined.withColumn(
        f"qerror",
        F.abs(F.col(f"{metric}_truth") - F.col(f"{metric}_dp"))
    )

    # Relative error |y - y~| / max(50, y)
    joined = joined.withColumn(
        f"relerror",
        F.col(f"qerror") /
        F.greatest(F.lit(50.0), F.col(f"{metric}_truth"))
    )

    return joined.select(
        run_col,
        *keys,
        f"{metric}_truth",
        f"{metric}_dp",
        f"qerror",
        f"relerror",
    )

tpch_query_results = {}

```

[8]: #Q1

```

# -----
# CONFIG
# -----
keys = ["l_returnflag", "l_linestatus"]
metrics = ["count_order"]
metric_casts = {"count_order": "long"} # important: count is integer

eps_val = 1

```

```

truth_path = "./output/no_dp/q1_result_csv"

# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "count_order")
    .withColumn("count_order", F.col("count_order").cast("long"))
)

# -----
# LOAD RUNS (ALL 1000)
# -----
runs_glob = f"./output/{OUTPUT}/q1/q1_dp_result_eps_{eps_val}_runs_*_csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "count_order")
    .withColumn("count_order", F.col("count_order").cast("long"))
)
print("Q1")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric="count_order",
    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

err_df.show(truncate=False)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),
    F.avg("relerror").alias("avg_relerror_per_run"),
)

```

```

).orderBy("run_id")

#per_run.show(20, truncate=False)
relererror = (
    err_df
    .select("relererror")
    .dropna()
    .toPandas()
)

qerror = (
    err_df
    .select("qerror")
    .dropna()
    .toPandas()
)

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_relererror_per_run").alias("mean_relererror_over_runs"),
)

tpch_query_results["Q1"] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_relererror_over_runs": float(overall.
    ↪collect()[0]["mean_relererror_over_runs"]),
    "qerror": qerror,
    "relererror": relererror
}

print(tpch_query_results["Q1"]["qerror"])

# Ensure numeric
#pdf["relererror"] = pd.to_numeric(pdf["relererror"])

# Plot
plt.figure(figsize=(5, 4))
plt.boxplot(tpch_query_results["Q1"]["qerror"], showliers=False)
plt.ylabel("relererror")
plt.title("relererror Distribution (1)")
plt.tight_layout()
plt.show()

```

Q1

Distinct runs = 1000

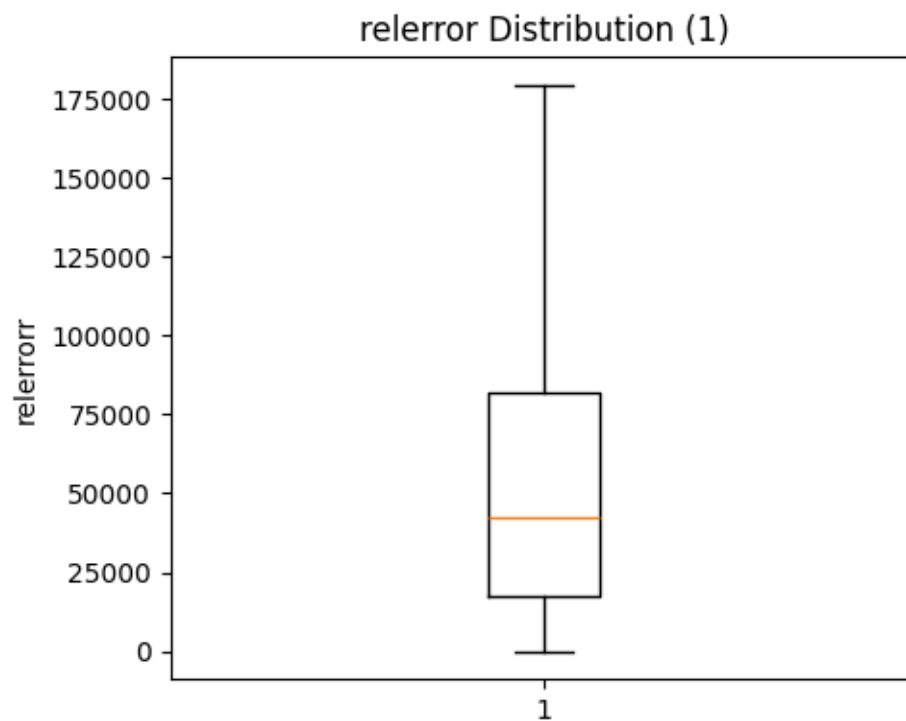
```
+-----+-----+-----+-----+-----+-----+-----+
-----+
|run_id|l_returnflag|l_linestatus|count_order_truth|count_order_dp|qerror
|reerror          |
+-----+-----+-----+-----+-----+-----+-----+
-----+
|670    |A          |F          |1478493.0      |1453903.0      |24590.0
|0.016631800082922272 |
|670    |A          |O          |0.0            |-154026.0
|154026.0|3080.52    |
|670    |N          |F          |38854.0        |-139321.0
|178175.0|4.585756936222783 |
|670    |N          |O          |2920374.0      |2843815.0      |76559.0
|0.02621547788057283 |
|670    |R          |F          |1478870.0      |1391196.0      |87674.0
|0.05928445367070804 |
|670    |R          |O          |0.0            |-142722.0
|142722.0|2854.44    |
|114    |A          |F          |1478493.0      |1560906.0      |82413.0
|0.05574121757762803 |
|114    |A          |O          |0.0            |-101328.0
|101328.0|2026.56    |
|114    |N          |F          |38854.0        |-123363.0
|162217.0|4.175039892932516 |
|114    |N          |O          |2920374.0      |2920347.0      |27.0
|9.245391172500508E-6 |
|114    |R          |F          |1478870.0      |1481570.0      |2700.0
|0.0018257182849067194|
|114    |R          |O          |0.0            |-45533.0       |45533.0
|910.66    |
|118    |A          |F          |1478493.0      |1517973.0      |39480.0
|0.026702865688237955 |
|118    |A          |O          |0.0            |-115156.0
|115156.0|2303.12    |
|118    |N          |F          |38854.0        |-126589.0
|165443.0|4.25806866731868 |
|118    |N          |O          |2920374.0      |2921753.0      |1379.0
|4.721997935880815E-4 |
|118    |R          |F          |1478870.0      |1525749.0      |46879.0
|0.03169920276968226 |
|118    |R          |O          |0.0            |191538.0
|191538.0|3830.76    |
|171    |A          |F          |1478493.0      |1412937.0      |65556.0
|0.04433974323855439 |
```

```
|171  |A      |0      |0.0      |-182520.0
|182520.0|3650.4  |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
-----+
```

only showing top 20 rows

```
      qerror
0      24590.0
1     154026.0
2     178175.0
3      76559.0
4      87674.0
...      ...
5995    49250.0
5996    38804.0
5997    29488.0
5998    43396.0
5999     2572.0
```

[6000 rows x 1 columns]



[9]: #Q4

```
# -----
```

```

# CONFIG
# -----
keys = ["o_orderpriority"]
metrics = "order_count"

eps_val = 1
truth_path = "./output/no_dp/q4_result_csv"
# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "order_count")
    .withColumn("order_count", F.col("order_count").cast("long"))
)

# -----
# LOAD RUNS (ALL 1000)
# -----
runs_glob = f"./output/{OUTPUT}/q4/q4_dp_result_eps_{eps_val}_runs_*_csv"
print(runs_glob)
dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "order_count")
    .withColumn("order_count", F.col("order_count").cast("long"))
)
print("Q4")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric=metrics,
    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

```

```

#err_df.show(truncate=False)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),
    F.avg("reerror").alias("avg_reerror_per_run"),
).orderBy("run_id")

#per_run.show(20, truncate=False)
reerror = (
    err_df
    .select("reerror")
    .dropna()
    .toPandas()
)

qerror = (
    err_df
    .select("qerror")
    .dropna()
    .toPandas()
)

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_reerror_per_run").alias("mean_reerror_over_runs"),
)

tpch_query_results["Q4"] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_reerror_over_runs": float(overall.
    ↪collect()[0]["mean_reerror_over_runs"]),
    "qerror": qerror,
    "reerror": reerror
}

#overall.show(truncate=False)

```

```
./output/supplier_policy_results/q4/q4_dp_result_eps_1_runs_*.csv
```

Q4

Distinct runs = 1000

```

[6]: #Q16

# -----
# CONFIG
# -----
keys = ["p_brand", "p_type", "p_size"]
metrics = "supplier_cnt"

eps_val = 1
truth_path = "./output/no_dp/q16_result_csv"
# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "supplier_cnt")
    .withColumn("supplier_cnt", F.col("supplier_cnt").cast("long")))
)

# -----
# LOAD RUNS (ALL 1000)
# -----
test_case="q16_DropExcess_1"
OUTPUT="supplier_policy_results"
runs_glob = f"./output/{OUTPUT}/{test_case}/
↳{test_case}_dp_result_eps_{eps_val}_runs_*.csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "supplier_cnt")
    .withColumn("supplier_cnt", F.col("supplier_cnt").cast("long")))
)
print("Q16")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,

```

```

        metric=metrics,
        run_col="run_id",
        join_type="left",
        truth_missing_as_zero=True,
    )

    #err_df.show(truncate=False)

    per_run = err_df.groupBy("run_id").agg(
        F.avg("qerror").alias("avg_qerror_per_run"),
        F.avg("reerror").alias("avg_reerror_per_run"),
    ).orderBy("run_id")

    reerror = (
        per_run
        .select("avg_reerror_per_run")
        .dropna()
        .toPandas()
    )

    qerror = (
        per_run
        .select("avg_qerror_per_run")
        .dropna()
        .toPandas()
    )

    overall = per_run.agg(
        F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
        F.avg("avg_reerror_per_run").alias("mean_reerror_over_runs"),
    )

    key=f'{OUTPUT}_{test_case}'
    with open(
        f"./output/{OUTPUT}/{key}.csv",
        "w"
    ) as f:
        f.write(key)
        f.write(",")
        f.write(",".join(str(x) for x in reerror.iloc[:, 0]))

    tpch_query_results[key] = {
        "mean_qerror_over_runs": float(overall.
        ↪collect()[0]["mean_qerror_over_runs"]),
        "mean_reerror_over_runs": float(overall.
        ↪collect()[0]["mean_reerror_over_runs"]),
        "qerror": qerror,
    }

```

```

        "relererror": relererror
    }

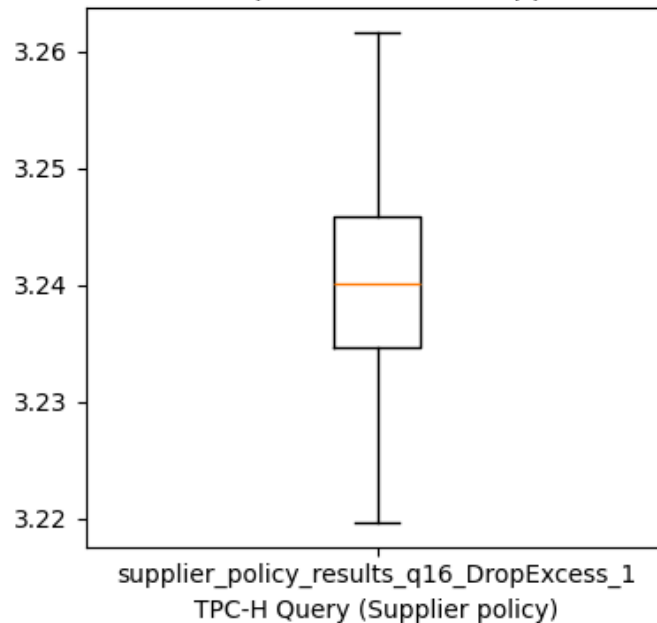
plt.figure(figsize=(4, 4))
plt.boxplot(relererror, showfliers=False)
plt.xticks([1], [key])
plt.xlabel("TPC-H Query (Supplier policy)")
plt.title("Relative Error for Q16 with OpenDP-Tumult: Supplier Policy, DropExcess(1)")
plt.tight_layout()
plt.show()

```

Q16

Distinct runs = 1000

Relative Error for Q16 with OpenDP-Tumult: Supplier Policy, DropExcess(1)



[7]: #Q16

```

# -----
# CONFIG
# -----

```

```

keys = ["p_brand", "p_type", "p_size"]
metrics = "supplier_cnt"

eps_val = 1
truth_path = "./output/no_dp/q16_result_csv"
# -----
# LOAD TRUTH
# -----
truth = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(truth_path)
    .select(*keys, "supplier_cnt")
    .withColumn("supplier_cnt", F.col("supplier_cnt").cast("long"))
)

# -----
# LOAD RUNS (ALL 1000)
# -----
test_case="q16_DropNonUnique"
OUTPUT="supplier_policy_results"
runs_glob = f"./output/{OUTPUT}/{test_case}/
↳{test_case}_dp_result_eps_{eps_val}_runs_*_csv"

dp_runs = (spark.read
    .option("header", "true")
    .option("inferSchema", "true")
    .csv(runs_glob)
    .withColumn(
        "run_id",
        F.regexp_extract(F.input_file_name(), r"runs_(\d+)_csv", 1).cast("int")
    )
    .select("run_id", *keys, "supplier_cnt")
    .withColumn("supplier_cnt", F.col("supplier_cnt").cast("long"))
)

print("Q16")
print("Distinct runs =", dp_runs.select("run_id").distinct().count())

err_df = compute_errors_privatesql_style(
    dp_df=dp_runs,
    truth_df=truth,
    keys=keys,
    metric=metrics,
    run_col="run_id",
    join_type="left",
    truth_missing_as_zero=True,
)

```

```

#err_df.show(truncate=False)

per_run = err_df.groupBy("run_id").agg(
    F.avg("qerror").alias("avg_qerror_per_run"),
    F.avg("reerror").alias("avg_reerror_per_run"),
).orderBy("run_id")

reerror = (
    per_run
    .select("avg_reerror_per_run")
    .dropna()
    .toPandas()
)

qerror = (
    per_run
    .select("avg_qerror_per_run")
    .dropna()
    .toPandas()
)

overall = per_run.agg(
    F.avg("avg_qerror_per_run").alias("mean_qerror_over_runs"),
    F.avg("avg_reerror_per_run").alias("mean_reerror_over_runs"),
)

key=f'{OUTPUT}_{test_case}'

with open(
    f"./output/{OUTPUT}/{key}.csv",
    "w"
) as f:
    f.write(key)
    f.write(",")
    f.write(",".join(str(x) for x in reerror.iloc[:, 0]))

tpch_query_results[key] = {
    "mean_qerror_over_runs": float(overall.
    ↪collect()[0]["mean_qerror_over_runs"]),
    "mean_reerror_over_runs": float(overall.
    ↪collect()[0]["mean_reerror_over_runs"]),
    "qerror": qerror,
    "reerror": reerror
}

```

```

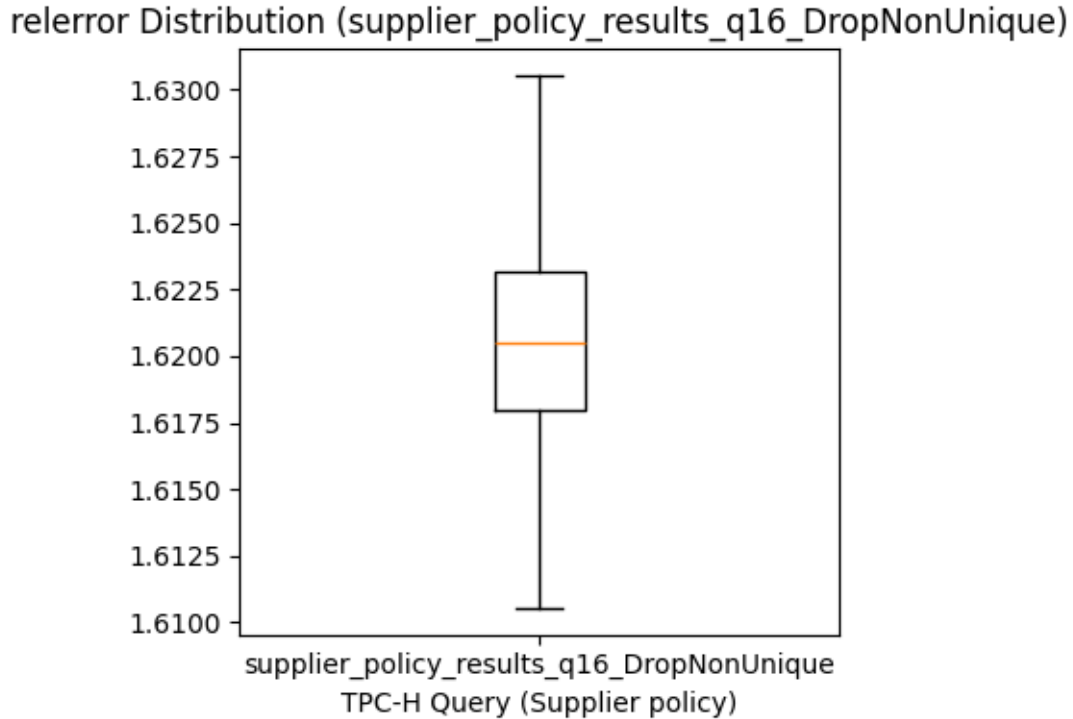
plt.figure(figsize=(4, 4))
plt.boxplot(reerror, showfliers=False)
plt.xticks([1], [key])
plt.xlabel("TPC-H Query (Supplier policy)")
plt.title("reerror Distribution " + f"({key})")
plt.tight_layout()
plt.show()

plt.figure(figsize=(2, 4))
plt.boxplot(reerror, showfliers=False)
plt.xticks([1], [r"$Q16_{S,DropNonUnique}$"])
plt.title("Relative Error for Q16 with \nOpenDP-Tumult")
plt.tight_layout()
plt.show()

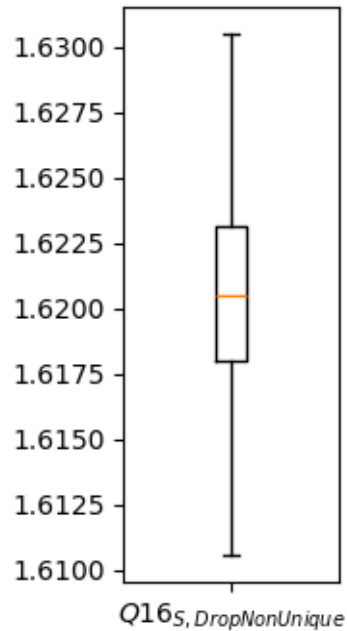
```

Q16

Distinct runs = 1000



Relative Error for Q16 with
OpenDP-Tumult



```
[10]: print("opendp eval with TPCB and supplier policy Results Summary:")

import matplotlib.pyplot as plt

# Use a stable list of labels (convert to strings) and numeric positions for
    ↳plotting
queries = list(tpch_query_results.keys())
queries_label = [str(q) for q in queries]
print("Queries evaluated:", queries_label)

avg_qerror = [tpch_query_results[q]["mean_qerror_over_runs"] for q in queries]
raw_qerror = [tpch_query_results[q]["qerror"] for q in queries]
print("Average Qerror per query:", avg_qerror)
avg_relerror = [tpch_query_results[q]["mean_relerror_over_runs"] for q in
    ↳queries]
print("Average RelError per query:", avg_relerror)

positions = list(range(len(queries_label)))

# Absolute error bar plot
fig, ax = plt.subplots(figsize=(4, 4))
```

```

ax.bar(positions, avg_qerror, color='tab:blue')
ax.set_xticks(positions)
ax.set_xticklabels(queries_label, rotation=45, ha='right')
ax.set_xlabel("TPC-H Query (Supplier policy)")
ax.set_ylabel("Average Absolute Error (Qerror)")
ax.set_title("Average Absolute Error Across Runs")
plt.tight_layout()
plt.show()

# Relative error bar plot
fig2, ax2 = plt.subplots(figsize=(4, 4))
ax2.bar(positions, avg_relerror, color='tab:orange')
ax2.set_xticks(positions)
ax2.set_xticklabels(queries_label, rotation=45, ha='right')
ax2.set_xlabel("TPC-H Query (Supplier policy)")
ax2.set_ylabel("Average Relative Error")
ax2.set_title("Average Relative Error Across Runs")
plt.tight_layout()
plt.show()

raw_qerror = [tpch_query_results[q]["qerror"] for q in queries]
raw_relerror = [tpch_query_results[q]["relerror"] for q in queries]
import numpy as np
raw_qerror_1d = [np.asarray(x).ravel() for x in raw_qerror]

plt.figure(figsize=(4, 4))
plt.boxplot(
    raw_qerror_1d,
    labels=queries_label,
    showfliers=False
)
plt.xticks(rotation=45, ha='right')
plt.xlabel("TPC-H Query (Supplier policy)")
plt.ylabel("Average Absolute Error (Qerror)")
plt.title("qerror Distribution Across Runs")
plt.tight_layout()
plt.show()

raw_relerror_1d = [np.asarray(x).ravel() for x in raw_relerror]

plt.figure(figsize=(4, 4))
plt.boxplot(
    raw_relerror_1d,
    labels=queries_label,
    showfliers=False
)
plt.xticks(rotation=45, ha='right')

```

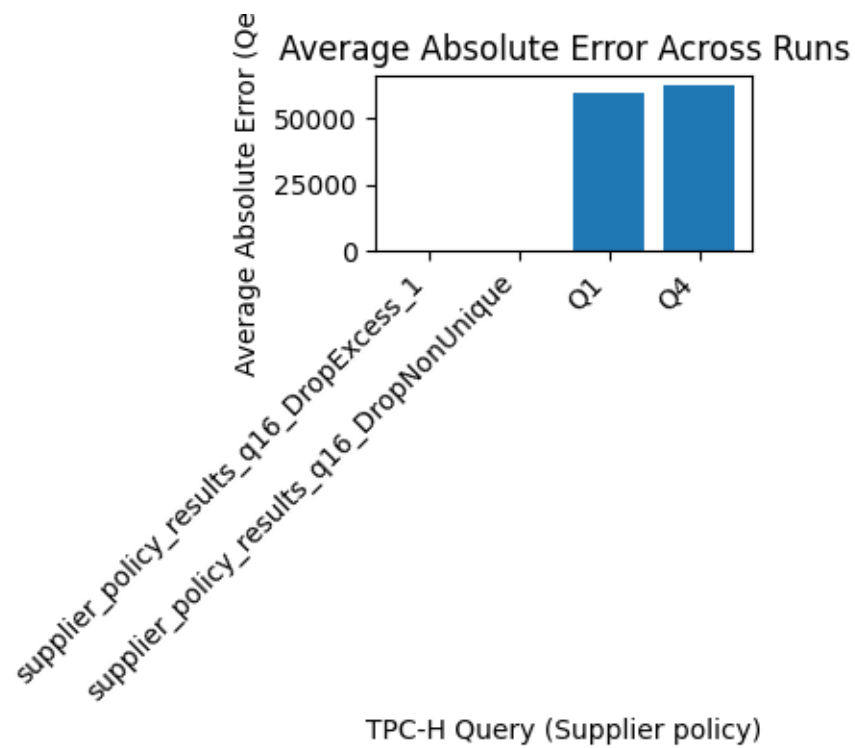
```
plt.xlabel("TPC-H Query (Supplier policy)")
plt.ylabel("Average Relative Error (RelError)")
plt.title("relerror Distribution Across Runs")
plt.tight_layout()
plt.show()
```

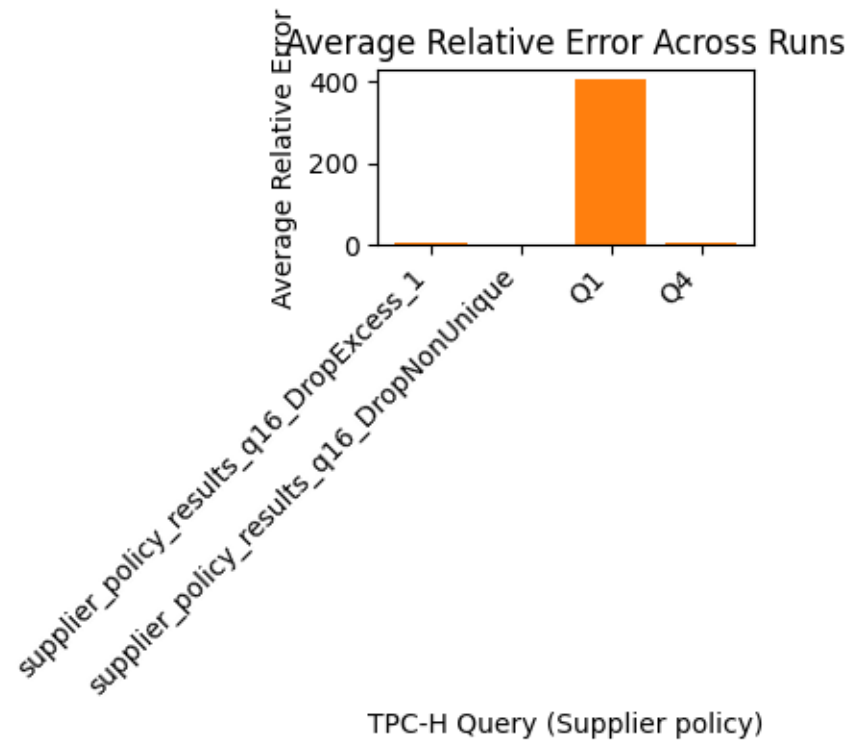
opendp eval with TPC-H and supplier policy Results Summary:

Queries evaluated: ['supplier_policy_results_q16_DropExcess_1',
'supplier_policy_results_q16_DropNonUnique', 'Q1', 'Q4']

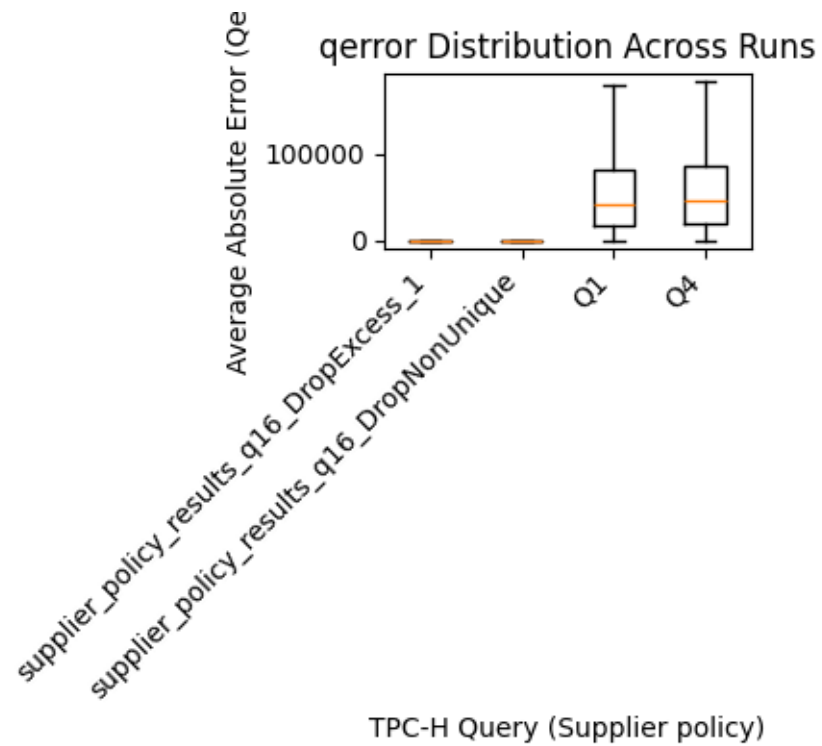
Average Qerror per query: [162.01907341333336, 81.02929799466676,
59884.262833333356, 62891.017799999914]

Average RelError per query: [3.2403814682666763, 1.620585959893278,
406.512966751318, 5.986769028723493]





```
/var/folders/xm/h9wx824x47311kz87v7c1g440000gn/T/ipykernel_2478/4221954323.py:46
: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
plt.boxplot(
```



```

/var/folders/xm/h9wx824x47311kz87v7c1g440000gn/T/ipykernel_2478/4221954323.py:61
: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been
renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be
dropped in 3.11.
plt.boxplot(

```

